

PAPER NAME

2408477_AnkitLalShresthaReport__part3
(1).pdf

AUTHOR

-

WORD COUNT

2909 Words

CHARACTER COUNT

16843 Characters

PAGE
COUNT

18 Pages

FILE SIZE

1.1 MB

SUBMISSION DATE

May 6, 2026 6:29 PM
GMT+5:45

REPORT DATE

May 6, 2026 6:29 PM GMT+5:45

● 15% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

- 6% Internet database
- 5% Publications database
- Crossref database
- Crossref Posted Content database
- 14% Submitted Works database

²⁹ Academic Year	Module	Assessment Number	Assessment Type
2026	6CS012- Artificial Intelligence and Machine Learning	1	Report

Title- Part 1 Report

<https://github.com/Ankitlal12/AI-MLModule.git>

¹¹ Student Id : 2408477
Student Name : Ankit Lal Shrestha
Section : L6CG4
Module Leader : Simon Giri
Tutor : Anish Khatiwada
Submitted on : 05/10/2026

Contents

Long Question.....	19 Error! Bookmark not defined.
Short Question.....	Error! Bookmark not defined.

Abstract

This report shows a sentiment classification analysis on financial news phrases on recurrent neural network architectures. This was aimed at classifying each sentence as negative, neutral or positive sentiment and comparing the behaviour of a Simple RNN, an LSTM and an LSTM with pre-trained word embeddings. A CSV dataset of 2,264 financial phrases was loaded to the notebook and, upon eliminating duplicates and any rows that were empty as a result of cleaning, it was left with 2,259 samples to model. The preprocessing of text involved lowercase letters, expansion of shortenings, URL deletion, mentions and hashtags, deletion of numbers and special characters, filtering of stopwords with a set of financial and negation words being retained, lemmatization, tokenization, and padding with percentiles. Adam optimizer, sparse categorical cross-entropy loss, class weights, early stopping, model checkpointing and learning-rate reduction are used to train three deep learning models. The LSTM that had pre-trained embeddings was the best with 81.12% test accuracy and 0.4974 test loss. The Simple RNN also was strong with an accuracy of 80.24, the trainable-embedding LSTM was also strong with 71.98% accuracy and could not learn a negative class effectively. The findings indicate that pre-trained embeddings enhanced the overall generalization, particularly the minority negative class, but the notebook version utilised pre-trained 50-dimensional GloVe vectors with Gensim instead of actually using Word2Vec embeddings. Financial numerical cues in future work should be maintained, minority-class handling be enhanced, and these recurrent models should be compared to finance-specific transformer models.

Keywords: sentiment analysis; financial text classification; RNN; LSTM; pre-trained embeddings; GloVe; NLP.

8 1. Introduction

Text classification is a fundamental NLP task whereby a model is used to assign a single or multiple predefined labels to a text. This task is particularly helpful in the financial sphere where brief statements regarding profit, sales, revenue, investment, acquisitions or market condition may impact investor sentiment and business decisions. The meaning of a sentence like operating profit increased is totally different to the meaning of a sentence like revenue decreased, although both are brief and similar in structure.

This project sought to construct an end-to-end sentiment classifier of financial phrases and compare recurrent architectures that can be used with sequential text data. The given notebook tries three options, which are Simple RNN with trainable embeddings, LSTM with trainable embeddings, and LSTM that uses a frozen pre-trained embedding matrix. This analogy is helpful since recurrent models are meant to process word order and context whereas pre-trained embeddings can inject richer semantic signals than a small task-specific dataset can tend to learn on its own.

The particular issue covered in this report is three-classification of English financial sentences into negative, neutral and positive. The models are compared regarding the data preparation, architecture, loss, class-wise precision/recall/F1-score, and training-time behavior. The report tracks the notebook and only reports values which are backed by the implementation outputs.

2. Dataset

The notebook used a CSV file named `financial_phrase.csv` containing labelled financial-news sentences. The loaded file had 2,264 rows and two columns: text and label. No missing values were present in either column. During preprocessing, five duplicate rows were removed, and rows whose cleaned text became empty were discarded, leaving 2,259 usable samples. The classification task is multi-class sentiment prediction with three labels: negative, neutral, and positive.

Dataset Property	Value from implementation
CSV file used	<code>financial_phrase.csv</code>
Initial number of rows	2,264
Columns	text, label
Missing values	0 in text; 0 in label
Duplicate rows	5
Rows after duplicate/empty-cleaned removal	2,259
Classes	negative, neutral, positive
Task type	Multi-class sentiment classification

Table 1. Summary of the dataset used in the implementation.

The data is imbalanced, with neutral sentences forming the majority class, followed by positive examples, while negative examples are the smallest class. This imbalance is important because minority-class learning becomes harder, especially for deep models trained on limited data. The notebook addressed this issue later through class weighting during training.

Class	Before cleaning	After cleaning
Neutral	1391	1386
Positive	570	570
Negative	303	303

Table 2. Class distribution before and after cleaning.

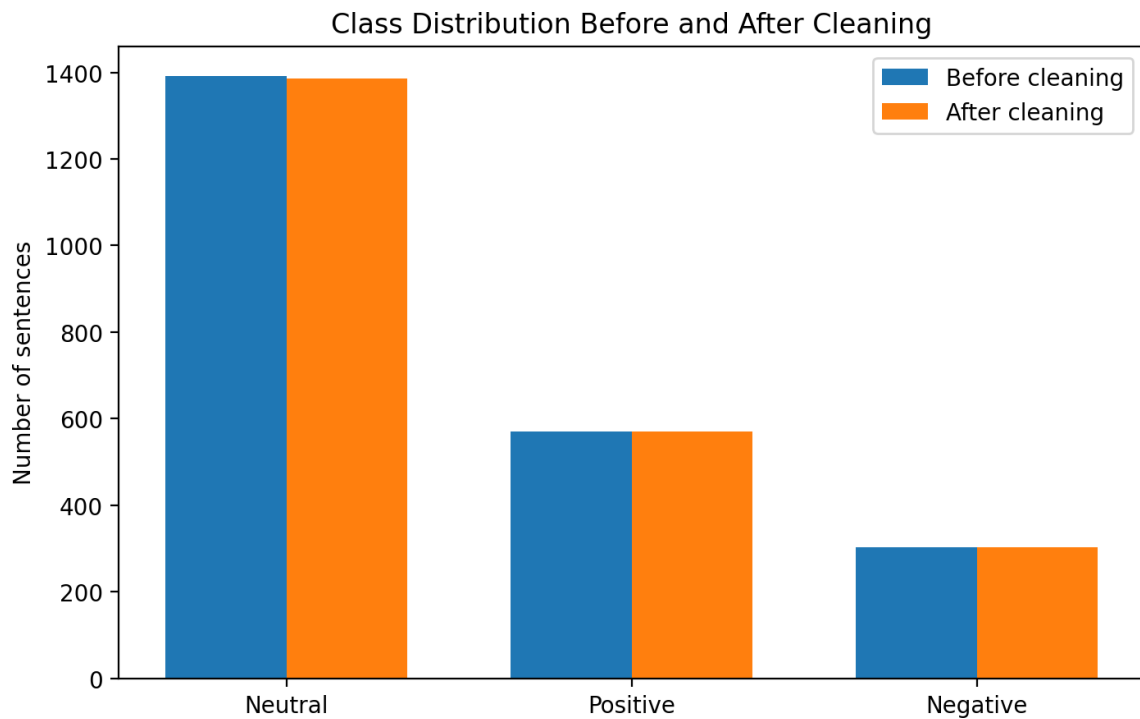


Figure 1. Class distribution before and after cleaning.

Basic descriptive statistics of the raw text show that the sentences are generally short to medium length. The mean sentence length is 121.96 characters and 22.44 words, while the median sentence length is 111 characters and 21 words. These statistics support the use of padded sequence models because the texts are short enough for compact sequence lengths but still contain enough variation to justify percentile-based padding rather than fixed truncation at a very high limit.

Statistic	Text length	Word count
Mean	121.96 characters	22.44 words
Standard deviation	56.24	10.15
Minimum	9	2
25th percentile	79	15
Median	111	21
75th percentile	157	28
Maximum	315	81

Table 3. Descriptive statistics of the raw sentences.

3. Methodology

3.1 Text Preprocessing

The text preprocessing pipeline followed the operations implemented in the notebook. Each sentence was lowercased and contractions were expanded so that forms such as “don’t” and “can’t” were normalized. URLs, mentions, hashtag symbols, punctuation, special characters, and numbers were removed. English stopwords were filtered, but important negation and finance-related words such as not, no, increase, decrease, rise, fall, profit, loss, gain, growth, and decline were deliberately preserved because they carry strong sentiment signals in financial language. Finally, WordNet lemmatization was applied to reduce inflected words to their base forms.

After cleaning, Keras Tokenizer ²⁶ was fitted on the training set only to avoid data leakage. Sequences were padded using the 95th percentile of training-sequence length, resulting in a maximum padded length of 22 tokens. This choice prevented a small number of longer sentences from making all samples unnecessarily long.

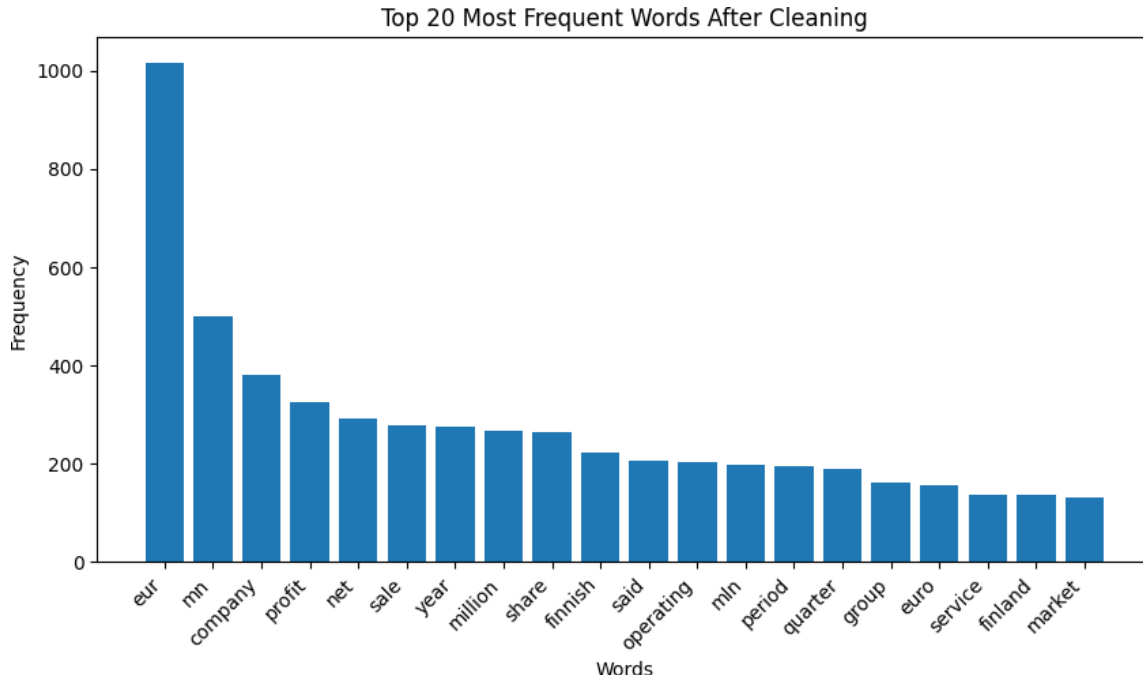


Figure 2. Top frequent words after text cleaning.

Step	Implementation detail
Label encoding	negative = 0, neutral = 1, positive = 2
Data split	70% training, 15% validation, 15% testing
Training samples	1,581
Validation samples	339
Testing samples	339
Vocabulary size	3,958
Maximum padded sequence length	22
Padded train shape	(1,581, 22)
Padded validation shape	(339, 22)
Padded test shape	(339, 22)

Table 4. Tokenization, label encoding and data split settings.

3.2 Model Architecture

Three neural network models were built. The first model used a trainable embedding layer followed by a SimpleRNN layer. The second replaced the recurrent layer with LSTM to better preserve longer dependencies and address vanishing-gradient issues. The third used an LSTM again, but replaced the trainable embedding layer with a frozen pre-trained embedding matrix of size (3,958, 50). The notebook loaded these vectors through Gensim from 50-dimensional GloVe embeddings. Therefore, while the project brief references Word2Vec-style embeddings, the actual implementation uses GloVe-based pre-trained word vectors.

Model	Architecture summary	Parameters
Simple RNN	Embedding(128) → SimpleRNN(64) → Dropout(0.3) → Dense(64, ReLU) → Dropout(0.3) → Dense(3, softmax)	523,331 trainable
LSTM	Embedding(128) → LSTM(64) → Dropout(0.3) → Dense(64, ReLU) → Dropout(0.3) → Dense(3, softmax)	560,387 trainable
LSTM + Pre-trained Embeddings	Frozen Embedding(50) → LSTM(64) → Dropout(0.3) → Dense(64, ReLU) → Dropout(0.3) → Dense(3, softmax)	231,695 total; 33,795 trainable

Table 5. Model architecture summary.

3.3 Loss Function, Optimizer and Hyperparameters

All three models were compiled with the Adam optimizer and sparse categorical cross-entropy loss. Sparse categorical cross-entropy was appropriate because the labels were integer-encoded rather than one-hot encoded. Accuracy was used as the main training metric. The models were trained for up to 15 epochs with a batch size of 32. EarlyStopping monitored validation loss with patience 3, ModelCheckpoint saved the best version of each model, and ReduceLROnPlateau reduced the learning rate when validation loss stopped improving.

To reduce the effect of class imbalance, class weights were computed from the training data. The negative class received the highest weight, followed by the positive class, while the majority neutral class received the smallest weight.

Class	Class weight
Negative	2.486
Neutral	0.543
Positive	1.321

Table 6. Class weights used to handle class imbalance.

The pre-trained embedding matrix covered 3,596 of the 3,958 vocabulary items, corresponding to 90.88% coverage. This relatively high coverage helps explain why the pre-trained embedding model generalized better than the LSTM that had to learn embeddings from scratch.

4. Experiments and Results

All models were evaluated on the same held-out test set of 339 samples. The notebook reports test accuracy, test loss, training time, classification reports, confusion matrices, and illustrative misclassifications. The following subsections align with the required report structure and keep the analysis grounded in the recorded notebook outputs.

4.1 RNN vs. LSTM Performance

The comparison between the Simple RNN and the trainable-embedding LSTM produced an interesting outcome. The Simple RNN achieved 80.24% test accuracy with 0.6339 loss, while the LSTM achieved only 71.98% accuracy with 0.7793 loss. In this notebook run, the more complex LSTM did not outperform the simpler recurrent model. The class-wise results show why: the LSTM completely failed to recover the negative class on the test set, giving it zero recall and zero F1-score. In contrast, the Simple RNN achieved 0.50 recall and 0.60 F1 for the negative class.

Model	Test Accuracy	Test Loss	Training Time (seconds)
Simple RNN	0.8024	0.6339	11.49
LSTM	0.7198	0.7793	10.83
LSTM + Pre-trained Embeddings	0.8112	0.4974	21.17

Table 7. Overall comparison of the three trained models.

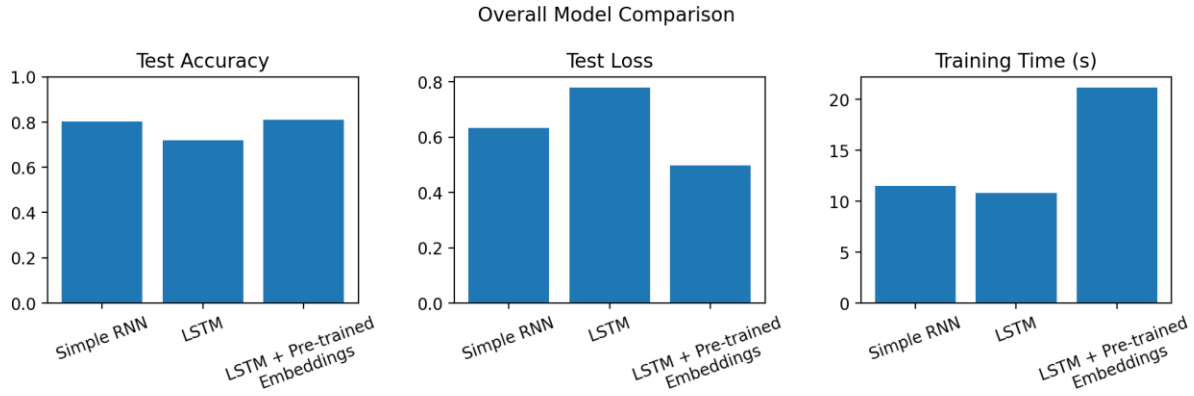


Figure 3. Overall comparison of test accuracy, loss, and training time.

4.2 Computational Efficiency

The notebook was executed in Google Colab and explicitly reported that GPU acceleration was available. However, the implementation did not log RAM or GPU-memory usage, so the computational-efficiency discussion must be limited to recorded training time and parameter counts. The LSTM with trainable embeddings was the fastest model in the notebook run at 10.83 seconds, followed closely by the Simple RNN at 11.49 seconds. The LSTM with pre-trained embeddings achieved the best predictive quality but required the longest training time in the recorded run at 21.17 seconds. This result shows that higher predictive performance did not coincide with the shortest training duration in this experiment.

Parameter counts provide additional context. The pre-trained embedding model had far fewer trainable parameters (33,795 trainable out of 231,695 total) than the two randomly initialized embedding models. Even so, wall-clock time was still higher in the notebook run, which suggests that model quality and measured training time should be considered separately.

Model	Macro F1	Weighted F1	Negative-class Recall
Simple RNN	0.73	0.80	0.50
LSTM	0.50	0.69	0.00
LSTM + Pre-trained Embeddings	0.73	0.81	0.52

Table 8. Comparison of imbalance-sensitive summary metrics.

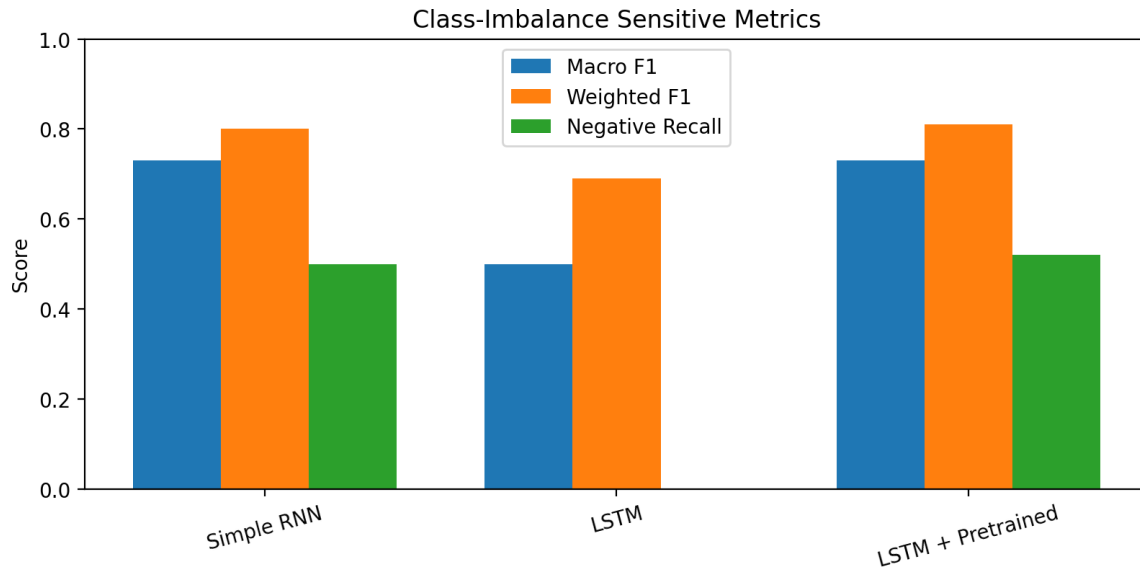


Figure 4. Macro F1, weighted F1, and negative-class recall across models.

4.3 Training with Different Embeddings

The notebook compared two embedding strategies: randomly initialized trainable embeddings in the Simple RNN and plain LSTM models, and a frozen pre-trained embedding matrix in the third model. The pre-trained model achieved the highest test accuracy (81.12%) and the lowest loss (0.4974), showing that external semantic knowledge improved generalization on this relatively small financial dataset. The difference is especially visible when comparing the two LSTM variants: replacing trainable embeddings with pre-trained embeddings raised accuracy by more than 9 percentage points and restored the model's ability to identify negative examples.

The per-class F1 comparison shows that pre-trained embeddings improved neutral-class performance slightly and greatly improved the negative class compared with the plain LSTM. The Simple RNN remained competitive overall, but the pre-trained embedding model provided the best balance across classes.

Model	Negative F1	Neutral F1	Positive F1
Simple RNN	0.60	0.87	0.72
LSTM	0.00	0.87	0.62
LSTM + Pre-trained Embeddings	0.57	0.91	0.71

Table 9. Per-class F1-score comparison across models.

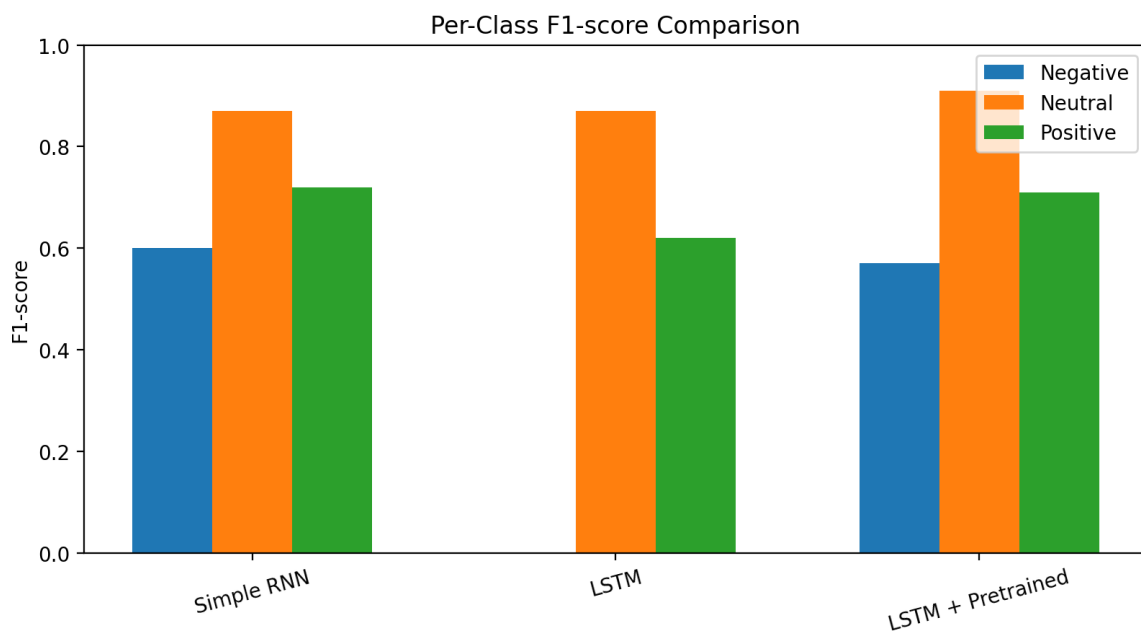


Figure 5. Per-class F1-score comparison across the three models.

4.4 Model Evaluation

Based on test accuracy, the best model was the LSTM with pre-trained embeddings. The model achieved balanced overall performance with 0.81 weighted F1-score and 0.73 macro F1-score. Its strongest class was neutral, while the negative class remained the most difficult.

Class	Precision	Recall	F1-score	Support
Negative	0.63	0.52	0.57	46
Neutral	0.93	0.88	0.91	208
Positive	0.64	0.79	0.71	85
Accuracy	-	-	0.81	339
Macro average	0.74	0.73	0.73	339
Weighted average	0.82	0.81	0.81	339

Table 10. Classification report for the best model: LSTM with pre-trained embeddings.

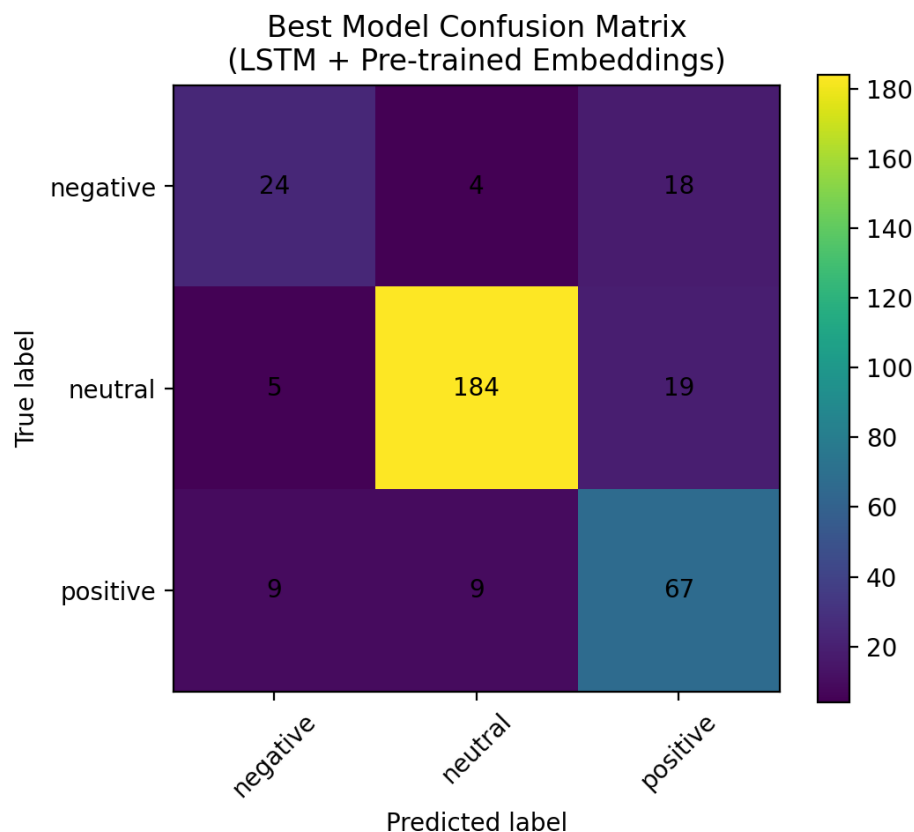
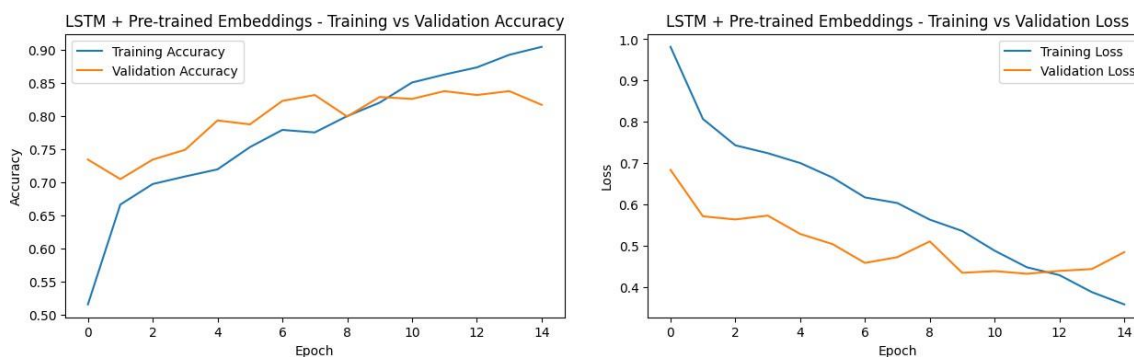


Figure 6. Confusion matrix for the best model.

The confusion matrix shows that the model classified neutral sentences most reliably, correctly predicting 184 of 208 neutral samples. It also performed reasonably well on positive examples, correctly identifying 67 of 85. The weakest area was the negative class, where only 24 of 46 examples were classified correctly. Many negative or neutral statements were confused with positive ones, suggesting that financially optimistic vocabulary sometimes dominated the model's interpretation.



23
Figure 7. Training and validation curves for the best model.

The training curves show a steady reduction in training loss and a generally improving validation trend. Validation accuracy peaked around the later epochs before flattening, while validation loss remained well below the starting point but began to fluctuate near the end. This behaviour indicates useful learning with some late-epoch overfitting pressure, making early stopping and learning-rate reduction appropriate.

4.5 Error Analysis

Error analysis was performed on the best model. The notebook reported 64 misclassified test examples. The mistakes reveal that subtle financial directionality and neutral statements with business terminology are particularly difficult. In several cases, words such as sales, profit, or company names appear in otherwise negative or neutral statements and seem to bias the model toward positive predictions.

Original sentence	Actual label	Predicted label
12 Sanoma News' advertising sales decreased by 2.0% year-on-year.	Negative	Positive
4 In the first quarter of 2008, Sacanfil's net sales totalled EUR million.	Neutral	Positive
12 Finnish paints and coatings company Tikkurila ...	Neutral	13 Positive
According to Gran, the company has no plans to move production...	Neutral	4 Positive
At the end of March 2007, the group's order book EUR mn up corresponding...	Positive	Neutral

Table 11. Examples of misclassified financial phrases from the test set.

These errors also highlight a limitation of the preprocessing strategy. Removing numbers reduces noise, but it can also remove important financial cues such as percentages, euro values, and directional changes that may be crucial for sentiment interpretation. Furthermore, the minority

negative class had fewer examples, ²⁰ which makes it harder for the model to learn robust negative patterns.

4.6 Real-time Prediction Interface

The notebook also implemented a Gradio-based real-time prediction interface. A user can enter a financial sentence, and the saved best model predicts the sentiment label together with class probabilities. This makes the trained model usable beyond notebook evaluation cells and demonstrates a simple deployment-oriented extension of the project.

Input sentence	Prediction	Comment
The company reported strong profit growth and higher sales.	Positive	Confidently correct prediction.
The firm announced a decline in revenue and lower operating profit.	Positive	Incorrect behaviour; negative wording still mapped to positive.
The company said the transaction is expected to close next month.	Neutral	Reasonable neutral prediction.

Table 12. Example real-time predictions from the notebook.

5. Conclusion and Future Work

²⁴ This project successfully implemented an end-to-end NLP pipeline for financial phrase sentiment classification. The dataset was cleaned, tokenized, padded, split into training/validation/testing subsets, and used to train three recurrent neural network models. The best model was the LSTM with pre-trained embeddings, which achieved 81.12% test accuracy and 0.4974 test loss on the held-out test set.

The experiments show three main findings. First, pre-trained embeddings improved overall model quality compared with trainable embeddings, especially when compared against the plain LSTM. Second, a more complex recurrent architecture did not automatically guarantee better performance, as the Simple RNN outperformed the trainable-embedding LSTM in this notebook run. Third, class imbalance remained a major challenge, with the negative class consistently harder to classify than neutral or positive phrases.

Several limitations remain. The notebook does not log memory consumption, so computational-efficiency analysis is restricted to timing and parameter counts. The preprocessing pipeline also removes numbers, which may suppress useful financial signals. In addition, although the coursework prompt mentions Word2Vec, the provided implementation actually uses pre-trained GloVe vectors through Gensim; this report preserves that distinction for accuracy.

Future work should explore stronger imbalance-handling strategies, retain selected numerical features, tune hyperparameters more extensively, and compare the recurrent models against domain-specific transformer models such as FinBERT. Testing finance-specific embeddings or contextual encoders would likely provide a stronger benchmark for subtle financial sentiment.

Appendix A: Key Notebook Outputs

Item	Output
TensorFlow version	2.19.0
GPU available	True
Embedding coverage	90.88%
Saved best model	best_financial_sentiment_model.keras
Tokenizer saved	tokenizer.pkl
Label encoder saved	label_encoder.pkl
Preprocessing config saved	preprocessing_config.pkl
GUI library	Gradio

Table. Table A1. Important implementation outputs recorded in the notebook.

● 15% Overall Similarity

Top sources found in the following databases:

- 6% Internet database
- 5% Publications database
- Crossref database
- Crossref Posted Content database
- 14% Submitted Works database

TOP SOURCES

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	University of Wolverhampton on 2025-05-23 Submitted works	1%
2	University of Greenwich on 2023-04-12 Submitted works	1%
3	mdpi.com Internet	1%
4	khuhub.khu.ac.kr Internet	<1%
5	UOW Malaysia KDU University College Sdn. Bhd on 2025-12-04 Submitted works	<1%
6	core.ac.uk Internet	<1%
7	Coventry University on 2026-04-07 Submitted works	<1%
8	"Conceptual Modeling", Springer Science and Business Media LLC, 2018 Crossref	<1%

9	University of Abertay Dundee on 2026-04-22 Submitted works	<1%
10	University of Nottingham on 2026-04-28 Submitted works	<1%
11	The University of Wolverhampton on 2024-05-14 Submitted works	<1%
12	BPP College of Professional Studies Limited on 2024-05-08 Submitted works	<1%
13	Dublin City University on 2026-04-20 Submitted works	<1%
14	University of Hertfordshire on 2022-04-29 Submitted works	<1%
15	mel.cgiar.org Internet	<1%
16	The Robert Gordon University on 2025-07-27 Submitted works	<1%
17	University of Arizona Global Campus (Canvas LTI 1.1) on 2025-12-03 Submitted works	<1%
18	arxiv.org Internet	<1%
19	All Saints Anglican School on 2026-03-05 Submitted works	<1%
20	Heriot-Watt University on 2024-11-28 Submitted works	<1%

21	University of Ulster on 2026-04-19 Submitted works	<1%
22	LTI 202223 on 2026-04-17 Submitted works	<1%
23	Universiti Teknologi MARA on 2026-01-25 Submitted works	<1%
24	University of Queensland on 2025-06-11 Submitted works	<1%
25	University of Surrey on 2024-09-01 Submitted works	<1%
26	University of Wolverhampton on 2026-05-05 Submitted works	<1%
27	Bal S. Virdee, Tanweer Ali, Jaume Anguera, Suman Lata Tripathy. "Con... Publication	<1%
28	University of Hertfordshire on 2025-04-28 Submitted works	<1%
29	University of Wolverhampton on 2021-02-15 Submitted works	<1%
30	H L Gururaj, Francesco Flammini, V Ravi Kumar, N S Prema. "Recent Tr... Publication	<1%
31	University of Hertfordshire on 2025-04-27 Submitted works	<1%